AI WeatherWise

Deliver Intelligent Weather Forecasts and Insights

Testing Phase Report

Team Details

Field	Details
Team ID	SWTID-2026-4935
Team Size	5
Team Leader	Rakshavi.V
Team Members	Raihana Fathima.S, Preetheeswari J, Priyanka S, Priyadarshini M
Testing Tool	Postman (API testing)

Contents

1. Introduction and objectives
2. Scope and test environment
3. Test strategy
4. Test cases
5. Test execution procedure
6. Defect log
7. Test summary and conclusion

1. Introduction and Objectives

The testing phase verifies that the AI WeatherWise backend works as specified: users can register and log in securely, manage favorite locations, retrieve weather data and receive AI-generated recommendations. All APIs are tested with Postman against the locally running server.

Objectives

- Verify each REST endpoint returns the correct status code and JSON response.
- Confirm authentication and authorization work, including rejection of missing or invalid JWT tokens.
- Validate input checking, sanitization and error handling.
- Confirm weather retrieval, caching and AI recommendation behave correctly, including fallback when an external key is unavailable.
- Confirm users can access only their own saved locations.

2. Scope and Test Environment

Item	Details
In scope	Authentication, favorite location CRUD, weather retrieval, AI recommendations, JWT middleware, validation, error handling, caching.
Out of scope	Frontend UI testing and load/stress testing (the backend is tested through Postman).
Backend	Node.js (v16+), Express.js, running locally (for example http://localhost:5001).
Database	MongoDB Atlas accessed through Mongoose.
External services	OpenWeatherMap API and Google Gemini AI (keys set in .env).
Client	Postman, with a Bearer token set in the Authorization header for protected routes.
Operating system	Windows 10/11, macOS or Linux.

3. Test Strategy

Test type	Purpose	Method
Functional	Each feature behaves as specified.	Send valid requests in Postman and compare with expected responses.
Authentication and security	Only authorized users reach protected data.	Call routes with no, invalid and valid tokens; confirm passwords are stored hashed in MongoDB.
Validation	Bad input is rejected.	Send missing fields, invalid email and malformed data.
Error handling	Failures return clear JSON errors.	Use an unknown city, stop an external dependency or remove an API key.

Test type	Purpose	Method
Integration	Routes, controllers, database and external APIs work together.	Run the end-to-end flow: register, login, save city, weather, AI recommendation.
Performance (basic)	Caching avoids repeat external calls.	Repeat the same weather request and compare response behavior.

4. Test Cases

Endpoint paths below follow the project convention; adjust them to match your route files. The Status column is left for you to record the actual result after running each test.

4.1 User authentication

ID	Scenario	Input / request	Expected result	Status
TC-01	Register with valid details	POST register: name, unique email, password	Success response; user saved; password stored as a bcrypt hash and not returned.	Pass/ Fail
TC-02	Register with existing email	POST register with an already used email	Error response; no duplicate user created.	Pass/ Fail
TC-03	Register with missing field	POST register without password	Validation error with a clear message.	Pass/ Fail
TC-04	Login with valid credentials	POST login: email, password	Success response containing a JWT token.	Pass/ Fail
TC-05	Login with wrong password	POST login with incorrect password	Authentication error; no token issued.	Pass/ Fail
TC-06	Login with unknown email	POST login with unregistered email	Authentication error; no token issued.	Pass/ Fail

4.2 JWT protection

ID	Scenario	Input / request	Expected result	Status
TC-07	Protected route without token	GET /api/favorites with no header	HTTP 401 Unauthorized.	Pass/ Fail
TC-08	Invalid or tampered token	Bearer token with altered characters	HTTP 401 Unauthorized.	Pass/ Fail
TC-09	Valid token	GET /api/favorites with Bearer token	Request accepted; user identified; 200 response.	Pass/ Fail

4.3 Favorite location management (CRUD)

ID	Scenario	Input / request	Expected result	Status
TC-10	Add a location	POST /api/favorites: name, city, state, country	Location created and linked to the logged-in user.	Pass/ Fail
TC-11	View saved locations	GET /api/favorites with token	Returns only the current user's locations.	Pass/ Fail
TC-12	Update a location	PUT /api/favorites/:id with new values	Location updated; updated data returned.	Pass/ Fail
TC-13	Delete a location	DELETE /api/favorites/:id	Location removed; later GET no longer lists it.	Pass/ Fail
TC-14	Access another user's location	Use user B's token on user A's location id	Access denied or not found; data not exposed.	Pass/ Fail
TC-15	Add with missing required field	POST /api/favorites without city	Validation error.	Pass/ Fail

4.4 Weather retrieval

ID	Scenario	Input / request	Expected result	Status
TC-16	Weather for valid city	GET /api/weather?city=Chennai	Temperature, humidity, wind and conditions returned.	Pass/ Fail
TC-17	Weather for invalid city	GET /api/weather?city=xyzabc	Clear error response; server does not crash.	Pass/ Fail
TC-18	Weather without city parameter	GET /api/weather	Validation error.	Pass/ Fail
TC-19	Repeated request (cache)	Same city requested twice	Second request served using cached data, if caching is enabled.	Pass/ Fail

4.5 AI recommendations

ID	Scenario	Input / request	Expected result	Status
TC-20	AI summary for valid city	AI recommendation request for Chennai	Plain-language summary and event-planning suggestions returned.	Pass/ Fail
TC-21	Adverse weather scenario	City with rain or strong wind	Suggestions include precautions or an indoor alternative.	Pass/ Fail
TC-22	Missing Gemini key (fallback)	Remove GEMINI_API_KEY, repeat request	Fallback response or clear error; server remains running.	Pass/ Fail
TC-23	Missing weather key (fallback)	Remove OPENWEATHER_API_KEY, request weather	Fallback data or clear error as designed.	Pass/ Fail

4.6 Security and input handling

ID	Scenario	Input / request	Expected result	Status
TC-24	Passwords hashed	Inspect the user document in MongoDB Atlas	Password field shows a bcrypt hash, never plain text.	Pass/ Fail
TC-25	Malicious input	Send script tags or operator characters in fields	Input is sanitized or rejected; no harmful data stored.	Pass/ Fail
TC-26	Secrets not exposed	Review responses and repository	No API keys, JWT secret or database URI in responses or committed files.	Pass/ Fail

5. Test Execution Procedure

Setup

```
npm install
npm start
# Console should show: MongoDB Connected: <host>
```

Steps

1. Create a Postman collection with folders for Auth, Favorites, Weather and AI.
2. Register a test user, then log in and save the returned JWT as a Postman environment variable.
3. Set the Authorization type to Bearer Token using that variable for protected requests.
4. Run the test cases in section 4 in order and record the actual result and status.
5. For each failure, log a defect (section 6), fix it, and run the test again.
6. Attach Postman screenshots as evidence.

Evidence

Item	Link / attachment
API testing video	API TESTING .mp4 (add link)
Register API screenshot	Insert here
Login API screenshot	Insert here
Weather testing screenshot	Insert here
Weather recommendation screenshot	Insert here

6. Defect Log

Record any problem found during testing using this table.

Defect ID	Test case	Description	Severity	Status
D-01				
D-02				

Defect ID	Test case	Description	Severity	Status
D-03				
D-04				

7. Test Summary and Conclusion

Metric	Count
Total test cases	26
Passed	____
Failed	____
Open defects	____

Testing confirms that the AI WeatherWise backend meets its functional and security requirements once all test cases pass and defects are closed. The verified features include secure registration and login with JWT, protected favorite-location management, real-time weather retrieval, AI-generated recommendations and consistent error handling. Complete the counts above after execution and update this conclusion to reflect the actual outcome.